

Name : Kanika Saini

Batch : B16

SAP ID : 590034098

Program : Btech CSE

Multiplication Without ‘ * ’ Operator

1. Problem Statement

Write an algorithm to find the product of two integers without using the multiplication (*) operator.

The solution must use repeated addition, but it should use the minimum possible number of additions.

For example:

$12 \times 3 \rightarrow$ add 12 only 3 times

$3 \times 12 \rightarrow$ add 12 only 3 times

$-9 \times -4 \rightarrow$ only 4 additions

Anything multiplied by 0 $\rightarrow$ 0 additions

These minimum-addition requirements are confirmed by the given test cases.

2. Mistakes in the Given Algorithm

The original algorithm in the image is essentially:

1. Define num1, num2, prod, index as integers

2. Get num1, num2
3. Set prod = 0, index = 1
4. Repeat while index <= num2
 - prod = prod + num1
 - index = index + 1
5. Print prod
6. Stop

Problems-

Mistake 1: It assumes num2 is positive.

If:

num1 = 6

num2 = -5

the loop does not execute and incorrectly gives 0 instead of -30.

Mistake 2: It does not correctly handle negative × negative.

Example: -9×-4 should give: 36

Mistake 3: It does not always use the minimum number of additions.

For: 3×12

the original algorithm performs 12 additions. But we can calculate:

$12 + 12 + 12 = 36$, using only 3 additions.

Mistake 4: It doesn't select the number having the smaller absolute value as the repetition count.

Mistake 5: It doesn't separately determine the sign of the final answer.

These issues matter because the assignment explicitly requires all four sign combinations, multiplication by zero, and minimum additions.

3. Corrected Algorithm

Use the number with the smaller absolute value as the smaller and the number with the larger absolute value as the larger to add.

Algorithm: Multiply Two Integers Without Using *

1. Start

2. Input num1 and num2

3. If num1 = 0 OR num2 = 0

 prod = 0

 Print prod

 Stop

4. Find absolute values:

 a = abs(num1)

 b = abs(num2)

5. If a < b

 smaller = a

 larger = b

Else

smaller = b

larger = a

6. Set:

prod = 0

index = 1

7. While index <= smaller

prod = prod + larger

index = index + 1

8. If exactly one of num1 and num2 is negative

prod = -prod

9. Print prod

10. Stop

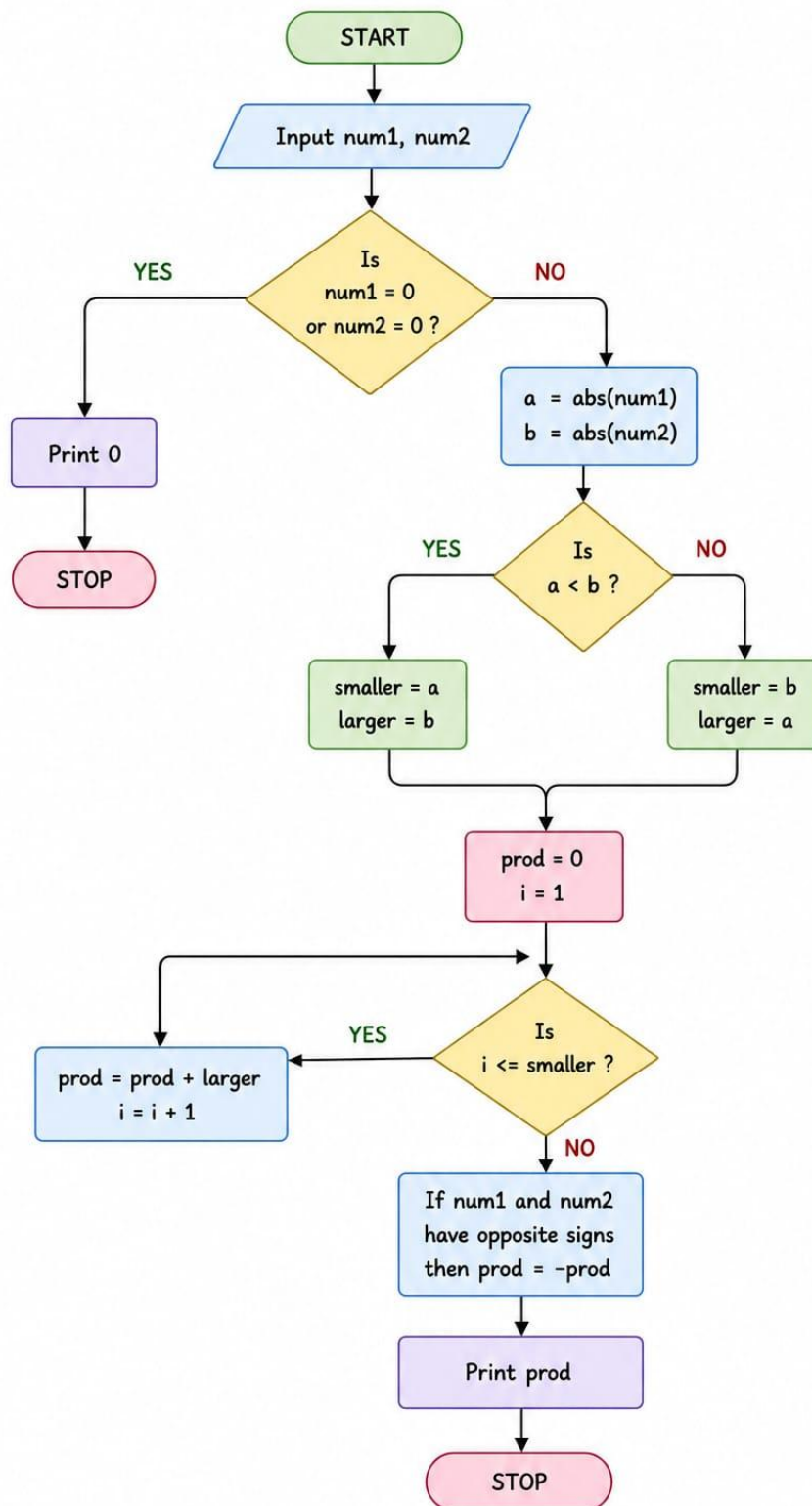
4. Mistakes in the Given Flowchart

The flowchart shown in the provided image has the same logical problems as the algorithm.

Errors

- It directly checks index <= num2.
- It doesn't handle a negative num2.
- It doesn't handle the sign of the final product correctly.
- It doesn't compare the absolute values of the two inputs.
- It may perform unnecessary additions.
- It doesn't explicitly handle zero before starting the loop.

5. Corrected Flowchart



6 . Verification of algorithm

Multiplication via Repeated Addition (with Sign Handling)								
Test	num1	num2	smaller x	larger x	Repeated Addition Process	Product	Additions	Verified
1	12	3	3	12	12+12+12	36	3	✓
2	3	12	3	12	12+12+12	36	3	✓
3	-6	5	5	6	6+6+6+6+6 → sign -	-30	5	✓
4	6	-5	5	6	6+6+6+6+6 → sign -	-30	5	✓
5	-9	-4	4	9	9+9+9+9	36	4	✓
6	-4	-9	4	9	9+9+9+9	36	4	✓
7	0	8	0	8	No addition	0	0	✓
8	8	0	0	8	No addition	0	0	✓
9	0	-7	0	7	No addition	0	0	✓
10	-7	0	0	7	No addition	0	0	✓
11	0	0	0	0	No addition	0	0	✓
12	1	-7	1	7	7 → sign -	-7	1	✓
13	-1	-9	1	9	9	9	1	✓

Conclusion:

The algorithm was verified using the test cases in the verification table. The results matched the expected output, showing that the algorithm works correctly for the given inputs. Hence, the algorithm is correct and successfully solves the problem.

Result:

The algorithm was implemented and verified successfully. The expected and actual outputs matched for all test cases.